

Modificación de Estilos CSS desde JavaScript

Introducción

JavaScript permite modificar dinámicamente los estilos CSS de los elementos HTML, lo que nos permite crear páginas web interactivas y responsivas. Existen varias formas de manipular los estilos desde JavaScript.

La Propiedad `style`

Acceso directo a la propiedad `style`

La forma más directa de modificar estilos es a través de la propiedad `style` de los elementos:

```
// Seleccionar un elemento
const elemento = document.getElementById('miElemento');

// Modificar estilos individuales
elemento.style.color = 'red';
elemento.style.backgroundColor = 'yellow';
elemento.style.fontSize = '20px';
elemento.style.border = '2px solid blue';
```



Nombres de propiedades CSS en JavaScript

Las propiedades CSS que contienen guiones se convierten a **camelCase** en JavaScript:

```
// CSS -> JavaScript
// background-color -> backgroundColor
```



```
// font-size -> fontSize
// border-radius -> borderRadius
// text-align -> textAlign

elemento.style.backgroundColor = 'lightblue';
elemento.style.fontSize = '16px';
elemento.style.borderRadius = '10px';
elemento.style.textAlign = 'center';
```

Modificar múltiples estilos

```
// Método 1: Uno por uno
elemento.style.width = '300px';
elemento.style.height = '200px';
elemento.style.background = 'linear-gradient(45deg, red, blue)';

// Método 2: Usando cssText
elemento.style.cssText = 'width: 300px; height: 200px; background: red';

// Método 3: Usando Object.assign
Object.assign(elemento.style, {
  width: '300px',
  height: '200px',
  backgroundColor: 'red',
  border: '1px solid black'
});
```

Trabajando con Classes CSS

La propiedad classList

Es más eficiente y mantenible trabajar con clases CSS predefinidas:

```
const elemento = document.getElementById('miElemento');

// Agregar una clase
elemento.classList.add('destacado');
```

```
// Agregar múltiples clases
elemento.classList.add('grande', 'azul', 'centrado');

// Quitar una clase
elemento.classList.remove('oculto');

// Alternar una clase (toggle) - si existe la quita, si no la añade
elemento.classList.toggle('activo');

// Verificar si tiene una clase
if (elemento.classList.contains('visible')) {
    console.log('El elemento es visible');
}

// Reemplazar una clase
elemento.classList.replace('viejo', 'nuevo');
```

Ejemplo práctico con classList

```
<!Doctype html>
<div id="caja" class="caja">Contenido</div>
<button onclick="cambiarEstilo()">Cambiar Estilo</button>
```



```
/* CSS */
.caja {
    width: 100px;
    height: 100px;
    background-color: lightgray;
    transition: all 0.3s ease;
}

.destacada {
    background-color: yellow;
    transform: scale(1.2);
    box-shadow: 0 0 10px rgba(0,0,0,0.5);
}

.grande {
    width: 200px;
```



```
height: 200px;
}
```

```
// JavaScript
function cambiarEstilo() {
  const caja = document.getElementById('caja');
  caja.classList.toggle('destacada');
  caja.classList.toggle('grande');
}
```



getComputedStyle()

Para obtener los estilos calculados (incluyendo CSS externo):

```
const elemento = document.getElementById('miElemento');

// Obtener todos los estilos computados
const estilos = window.getComputedStyle(elemento);

// Obtener propiedades específicas
const color = estilos.getPropertyValue('color');
const fontSize = estilos.getPropertyValue('font-size');
const backgroundColor = estilos.backgroundColor;

console.log('Color:', color);
console.log('Tamaño de fuente:', fontSize);
console.log('Color de fondo:', backgroundColor);
```



Diferencias entre style y getComputedStyle

```
const elemento = document.getElementById('miElemento');

// style: solo devuelve estilos inline
console.log(elemento.style.color); // Puede estar vacío

// getComputedStyle: devuelve el estilo final computado
```



```
const estilosComputados = getComputedStyle(elemento);  
console.log(estilosComputados.color); // Siempre tiene valor
```

Manipulación de Hojas de Estilo

Con JavaScript, también es posible acceder y modificar hojas de estilo completas.

Acceder a las hojas de estilo

Para acceder a las hojas de estilo cargadas en el documento:

```
// Obtener todas las hojas de estilo  
const hojas = document.styleSheets;  
  
// Acceder a una hoja específica  
const primeraHoja = document.styleSheets[0];  
  
// Obtener reglas de una hoja  
const reglas = primeraHoja.cssRules || primeraHoja.rules;
```



Añadir reglas CSS dinámicamente

También podemos añadir nuevas reglas CSS:

```
// Crear una nueva hoja de estilos  
const style = document.createElement('style');  
document.head.appendChild(style);  
const sheet = style.sheet;  
  
// Añadir reglas CSS  
sheet.insertRule('.nueva-clase { color: red; font-size: 20px; }', 0);  
sheet.insertRule('@media (max-width: 600px) { .responsive { display: r
```



Mejores Prácticas

Rendimiento

```
// ❌ Malo: Múltiples modificaciones del DOM
```

```
elemento.style.width = '100px';  
elemento.style.height = '100px';  
elemento.style.background = 'red';
```

```
// ✅ Bueno: Una sola modificación
```

```
elemento.style.cssText = 'width: 100px; height: 100px; background: red';
```

```
// ✅ Mejor: Usar clases CSS
```

```
elemento.classList.add('mi-estilo');
```



Mantenibilidad

```
// ❌ Malo: Estilos hardcoded en JavaScript
```

```
elemento.style.color = 'red';  
elemento.style.fontSize = '16px';
```

```
// ✅ Bueno: Usar clases CSS definidas
```

```
elemento.classList.add('texto-destacado');
```

